MEMORIA TÉCNICA

PROYECTO 1: VIDEOTECA RETRO

CE en Desarrollo de aplicaciones en lenguaje Python

Curso 2025–2026

Sumario

1. Descripción del sistema.....2
2. Arquitectura de software (POO avanzada).....3
2.1. Jerarquía de clases.....3
2.2. Encapsulación y validación de lógica de negocio.....4
3. Persistencia de datos y modelo relacional.....5
3.1. Diseño del esquema SQL.....5
3.2. Integridad de las consultas.....6
4. Interfaz web con Django.....7
4.1. Estructura del proyecto Django.....7
4.2. Vistas y rutas.....8
4.3. Seguridad y validación de entradas.....9
4.4. Plantillas HTML.....10
5. Capa de inteligencia de datos.....12
5.1. Limpieza y normalización de datos.....12
5.2. Visualización y hallazgos.....13
6. Calidad y verificación.....14
6.1. Batería de pruebas unitarias.....14
6.2. Gestión de validación en la interfaz de consola.....15
7. Conclusiones y líneas de mejora.....16

1. Descripción del sistema

El sistema nace de la necesidad real de un coleccionista de videojuegos retro de gestionar un inventario creciente de forma ordenada, segura y accesible. Las soluciones genéricas de gestión de colecciones no contemplan la particularidad del ecosistema retro: la coexistencia de soportes físicos (cartuchos, discos, casetes) con copias digitales distribuidas en plataformas de descarga, cada uno con sus propias necesidades de catalogación y localización. Esta aplicación resuelve exactamente esa brecha.

El modelo de datos distingue de forma nativa entre dos tipos de ítem: el **JuegoFisico**, que almacena la ubicación física en estantería para facilitar su localización inmediata dentro de una colección voluminosa; y el **JuegoDigital**, que registra el enlace o referencia de descarga asociada. Ambos tipos comparten un contrato común definido por la clase abstracta **Item**, lo que garantiza consistencia en la gestión independientemente del formato.

El núcleo funcional implementa el ciclo de vida completo de los datos mediante las cuatro operaciones **CRUD**. La creación de registros permite añadir títulos con todos sus metadatos relevantes (plataforma, año, género, formato y ubicación o enlace). La lectura presenta la colección en una tabla organizada por plataformas y enriquecida con el estado actual de préstamo de cada título. La actualización permite corregir o ampliar cualquier campo de un registro existente. La eliminación incluye una pantalla de confirmación explícita: evita borrados accidentales y gestiona automáticamente la integridad referencial eliminando también el préstamo activo si lo hubiera.

Complementariamente, la aplicación incluye un módulo de gestión de préstamos. Cuando un título se cede a un tercero, el sistema registra el nombre del destinatario y la fecha de inicio del préstamo. La restricción de clave primaria compartida entre las tablas juegos y prestamos garantiza a nivel de base de datos que un mismo juego no pueda tener dos préstamos activos simultáneos. La devolución se registra con un único clic, devolviendo el título al estado de disponible en biblioteca.

Desde el punto de vista técnico, el proyecto representa una evolución deliberada en tres frentes. En cuanto a la persistencia, se ha migrado desde archivos planos **JSON**, volátiles y sin integridad referencial, hacia una base de datos relacional **SQLite**, que ofrece transacciones **ACID**, consultas **JOIN** y validación de tipos sin necesidad de infraestructura externa. En cuanto a la interfaz, se ha superado la limitación de la consola de texto mediante una aplicación web construida con **Django**, accesible desde cualquier navegador. Por último, en cuanto a la inteligencia sobre los datos, se ha incorporado una capa de análisis exploratorio basada en **Pandas** y **Matplotlib** que transforma el inventario en información accionable: distribución por plataforma, evolución temporal de la colección y correlación histórica entre géneros y décadas.

2. Arquitectura de software (POO avanzada)

La aplicación ha sido diseñada íntegramente bajo el paradigma de Programación Orientada a Objetos (POO), aplicando de forma deliberada los tres pilares fundamentales: herencia, polimorfismo y encapsulación. Esta decisión arquitectónica no es arbitraria: permite que el sistema sea extensible en el futuro (por ejemplo, añadir nuevos tipos de soporte como **JuegoEnNube**) sin necesidad de modificar el código existente, cumpliendo así el principio abierto/cerrado de SOLID.

2.1. Jerarquía de clases

La jerarquía se organiza en torno a una clase base abstracta de la que heredan todas las especializaciones:

- **Clase base abstracta (`Item`):** Define y encapsula los cuatro atributos comunes a todo videojuego (título, plataforma, año y género), y establece el contrato polimórfico mediante el método abstracto **`mostrar_detalle()`**. Al declarar **`Item`** como **`ABC`** (Abstract Base Class), Python impide instanciarla directamente, garantizando que solo se puedan crear objetos de los tipos concretos. Esto asegura que cualquier elemento del catálogo tenga siempre una implementación específica de visualización.
- **`JuegoFisico`:** Extiende la clase base añadiendo el atributo **`estanteria`**, que registra la ubicación física del título dentro de la colección (por ejemplo, **`"Estante A3"`**). Este campo es fundamental en colecciones voluminosas donde la localización manual sin referencia resulta inviable. Su implementación de **`mostrar_detalle()`** devuelve el prefijo **`[FÍSICO]`** junto a la estantería asignada.
- **`JuegoDigital`:** Se especializa en la gestión del acceso lógico, almacenando en el atributo **`enlace`** la URL o referencia de descarga asociada al título. Su **`mostrar_detalle()`** devuelve el prefijo **`[DIGITAL]`** y el enlace correspondiente. La distinción entre ambas subclases permite que la interfaz web muestre información relevante para cada formato sin condicionales dispersos por el código.

2.2. Encapsulación y validación de lógica de negocio

Todos los atributos de las clases se han definido como privados mediante el prefijo de doble guion bajo **(__)**, lo que activa el *name mangling* de Python e impide el acceso directo desde el exterior. La exposición controlada de cada atributo se realiza exclusivamente a través de decoradores **`@property`**, que actúan como punto único de acceso y permiten añadir lógica de validación sin romper la interfaz pública de la clase.

La validación más relevante es la del año de lanzamiento, realizada en el método privado **`_validar_anio()`**, invocado automáticamente desde el constructor **`__init__()`** en el momento de crear cualquier objeto:

```
def _validar_anio(self, anio):
    if anio > datetime.now().year:
        raise ValueError("El año no puede ser superior al actual.")
    return anio
```

Esta validación actúa como primera línea de defensa a nivel de modelo, independientemente de por dónde llegue el dato (consola, formulario web o script de carga masiva). En caso de dato inválido, se lanza un **`ValueError`** con un mensaje descriptivo que la capa superior puede capturar y presentar al usuario. En la interfaz web Django, las vistas envuelven la creación y edición de juegos en unos bloques **`try-except`**, de forma que un año incorrecto se traduce en un mensaje de error visible en el formulario sin interrumpir la aplicación ni generar una página de error **HTTP 500**.

3. Persistencia de datos y modelo relacional

El sistema utiliza SQLite3 como motor de base de datos relacional, una elección técnica justificada por tres razones: no requiere proceso servidor externo (el archivo **`.db`** es autocontenido), está incluido en la biblioteca estándar de Python sin dependencias adicionales, y ofrece soporte completo de SQL con transacciones ACID, lo que garantiza integridad de los datos incluso ante interrupciones inesperadas del sistema.

3.1. Diseño del esquema SQL

Se ha diseñado un esquema relacional mínimo pero completo que modela las dos entidades principales del dominio: el inventario de juegos y el registro de préstamos activos. La decisión de separar ambas en tablas distintas, en lugar de añadir columnas de préstamo directamente a **`juegos`**, responde a un principio de diseño relacional: los préstamos son una entidad independiente con su propio ciclo de vida, y su ausencia (juego disponible) es tan válida como su presencia.

```sql
-- Tabla principal de inventario
CREATE TABLE IF NOT EXISTS juegos (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo     TEXT    NOT NULL,
    plataforma TEXT    NOT NULL,
    anio       INTEGER NOT NULL,
    genero     TEXT    NOT NULL,
    tipo       TEXT    NOT NULL,  -- "Físico" o "Digital"
    extra      TEXT               -- Estantería o URL de descarga
);

-- Tabla de gestión de préstamos (relación 1:1 activa)
CREATE TABLE IF NOT EXISTS prestamos (
    juego_id     INTEGER PRIMARY KEY,  -- FK y PK a la vez
    destinatario TEXT    NOT NULL,
    fecha_inicio TEXT,
    FOREIGN KEY (juego_id) REFERENCES juegos(id)
);
```

El diseño más relevante es el de la tabla **`prestamos`**. En lugar de usar un **`id`** propio con **`AUTOINCREMENT`**, se ha elegido **`juego_id`** como **`PRIMARY KEY`** directamente. Esta decisión tiene una consecuencia semántica importante: **a nivel de motor de base de datos, es imposible insertar dos filas con el mismo `juego_id`**, lo que significa que la restricción de "un juego solo puede estar prestado a una persona a la vez" no depende de la lógica de la aplicación, sino que está garantizada por el esquema. Si se intentara registrar un segundo préstamo para el mismo título, SQLite lanzaría un **`IntegrityError`** que el código captura y presenta como mensaje de error al usuario.

La columna **`fecha_inicio`** registra la fecha en formato **`YYYY-MM-DD`** generada automáticamente en el momento del préstamo mediante **`datetime.now().strftime("%Y-%m-%d")`**, proporcionando trazabilidad histórica de cuándo se cedió cada título.

3.2. Integridad de las consultas

Toda la interacción con la base de datos se centraliza en la clase **`DatabaseManager`**, que actúa como única puerta de entrada al almacenamiento. Ninguna otra capa del sistema ejecuta SQL directamente. Todas las consultas utilizan exclusivamente parámetros posicionales **(?)** en lugar de interpolación de cadenas, eliminando por diseño cualquier posibilidad de inyección SQL:

```
self.conn.execute(query, (titulo, plataforma, anio, genero, tipo, extra))
```

La recuperación de la colección completa se realiza mediante un **`LEFT JOIN`** que, en una única consulta eficiente, combina los datos del inventario con el estado de préstamo de cada título:

```
SELECT j.id, j.titulo, j.plataforma, j.anio, j.genero,
       j.tipo, j.extra, p.destinatario
FROM juegos j
LEFT JOIN prestamos p ON j.id = p.juego_id
ORDER BY j.plataforma, j.titulo
```

El uso de **`LEFT JOIN`** en lugar de **`INNER JOIN`** es una decisión deliberada: garantiza que los juegos disponibles (sin fila en **`prestamos`**) también aparezcan en el resultado, con `NULL` en la columna **`destinatario`**. La interfaz web interpreta ese **`NULL`** para mostrar el estado "En biblioteca" o "Prestado a: nombre" según corresponda. El resultado se ordena por plataforma y título, de forma que la presentación tabular agrupa visualmente los títulos por sistema sin necesidad de postprocesado en Python.

4. Interfaz web con Django

La capa de interacción con el usuario se ha implementado mediante **Django**, el framework web más consolidado del ecosistema Python. La elección de Django frente a alternativas más ligeras como **Flask** responde a su filosofía *"batteries included"*: proporciona de serie un sistema de plantillas, gestión de rutas, protección CSRF, mensajes flash y administración de archivos estáticos, reduciendo el código boilerplate y permitiendo centrarse en la lógica de negocio. La interfaz web sustituye y supera a la anterior consola de texto, ofreciendo una experiencia visual accesible desde cualquier navegador de la red local sin necesidad de instalar software adicional en el equipo del usuario.

4.1. Estructura del proyecto Django

Django organiza el código en dos niveles: el **proyecto** (configuración global) y las **aplicaciones** (módulos funcionales). La estructura adoptada es:

- **`videoteca/`** — Módulo de configuración global del proyecto. Contiene **`settings.py`**, donde se declaran las aplicaciones instaladas, la base de datos y la zona horaria (**`Europe/Madrid`**); y **`urls.py`**, que actúa como enrutador raíz delegando todas las rutas de usuario a la aplicación **`catalogo`**.

- **`catalogo/`** — Aplicación principal que concentra toda la lógica funcional: las vistas (**`views.py`**), las rutas propias (**`urls.py`**) y las plantillas HTML. Al separar la lógica en una aplicación independiente del proyecto, el código queda desacoplado y sería reutilizable en un proyecto Django más amplio sin modificaciones.

- **`catalogo/templates/catalogo/`** — Directorio donde Django busca las plantillas HTML. La doble anidación (**`catalogo/catalogo/`**) es una convención de Django para evitar colisiones de nombres entre aplicaciones distintas que pudieran tener plantillas con el mismo nombre.

La aplicación no utiliza los modelos ORM de Django para la base de datos, sino que reutiliza directamente la clase **`DatabaseManager`** ya existente, que gestiona la conexión SQLite nativa. Esto garantiza compatibilidad total con la base de datos generada por el script **`generar_datos.py`** y con los tests unitarios existentes.

4.2. Vistas y rutas

Django sigue el patrón arquitectónico **MTV (Model-Template-View)**, análogo al MVC clásico: la vista recibe la petición HTTP, consulta los datos a través de **`DatabaseManager`**, y devuelve una respuesta renderizando una plantilla con esos datos. Cada operación del sistema tiene su propia vista en **`catalogo/views.py`** y su ruta registrada en **`catalogo/urls.p`**y:

```
urlpatterns = [
    path('',                    views.coleccion,      name='coleccion'),
    path('añadir/',             views.añadir_juego,   name='añadir_juego'),
    path('editar/<int:id>/',    views.editar_juego,   name='editar_juego'),
    path('eliminar/<int:id>/', views.eliminar_juego, name='eliminar_juego'),
    path('prestar/<int:id>/',   views.prestar_juego,  name='prestar_juego'),
    path('devolver/<int:id>/', views.devolver_juego, name='devolver_juego'),
    path('analisis/',           views.analisis,       name='analisis'),
]
```

El segmento **`<int:id>`** en las rutas de edición, borrado, préstamo y devolución es un **conversor de tipo** de Django: extrae automáticamente el identificador numérico de la URL y lo pasa como argumento entero a la vista, eliminando la necesidad de conversiones manuales y previniendo que se invoquen las vistas con IDs no numéricos. Cada ruta tiene un **`name`** que permite referenciarla desde las plantillas mediante **`{% url 'nombre' %}`**, desacoplando las URLs del HTML y facilitando cambios futuros en la estructura de rutas sin tocar ninguna plantilla.

Las vistas que modifican datos (añadir, editar, eliminar, prestar, devolver) siguen el patrón **POST/Redirect/GET**: tras procesar con éxito un formulario POST, redirigen al usuario mediante **`redirect()`** en lugar de renderizar directamente. Esto evita la duplicación de operaciones si el usuario recarga la página tras un envío.

4.3. Seguridad y validación de entradas

La seguridad se ha tratado como un requisito transversal, no como un añadido posterior. Se han implementado cuatro capas de protección complementarias:

- **Token CSRF:** Django genera automáticamente un token único por sesión que se incrusta en cada formulario mediante **`{% csrf_token %}`**. Toda petición POST sin token válido es rechazada con un error 403 antes de llegar a la vista. Esto protege contra ataques de *Cross-Site Request Forgery*, en los que una página maliciosa podría intentar ejecutar acciones en nombre del usuario autenticado.
- **Validación en servidor:** Las vistas comprueban que ningún campo obligatorio llegue vacío y que el año sea un número entero antes de interactuar con la base de datos. La validación se realiza siempre en el servidor, independientemente de los atributos HTML del formulario **(`required`, `type="number"`)**, ya que las restricciones del lado cliente pueden ser fácilmente eludidas.
- **Consultas parametrizadas:** El **`DatabaseManager`** utiliza exclusivamente marcadores de posición **(?)** en todas las sentencias SQL. Los valores nunca se concatenan directamente en la cadena de la consulta, lo que elimina por diseño cualquier posibilidad de inyección SQL, independientemente del contenido que introduzca el usuario.
- **Confirmación de borrado:** La eliminación de un registro es una operación irreversible. Para prevenirla de forma accidental, se ha implementado un flujo de dos pasos: la petición **`GET`** a **`/eliminar/<id>/`** muestra una página de confirmación con el nombre del juego afectado; solo una petición **`POST`** explícita desde esa misma página ejecuta el borrado definitivo. Esto impide también que un simple enlace externo o un bot de indexación pueda provocar eliminaciones involuntarias.

4.4. Plantillas HTML

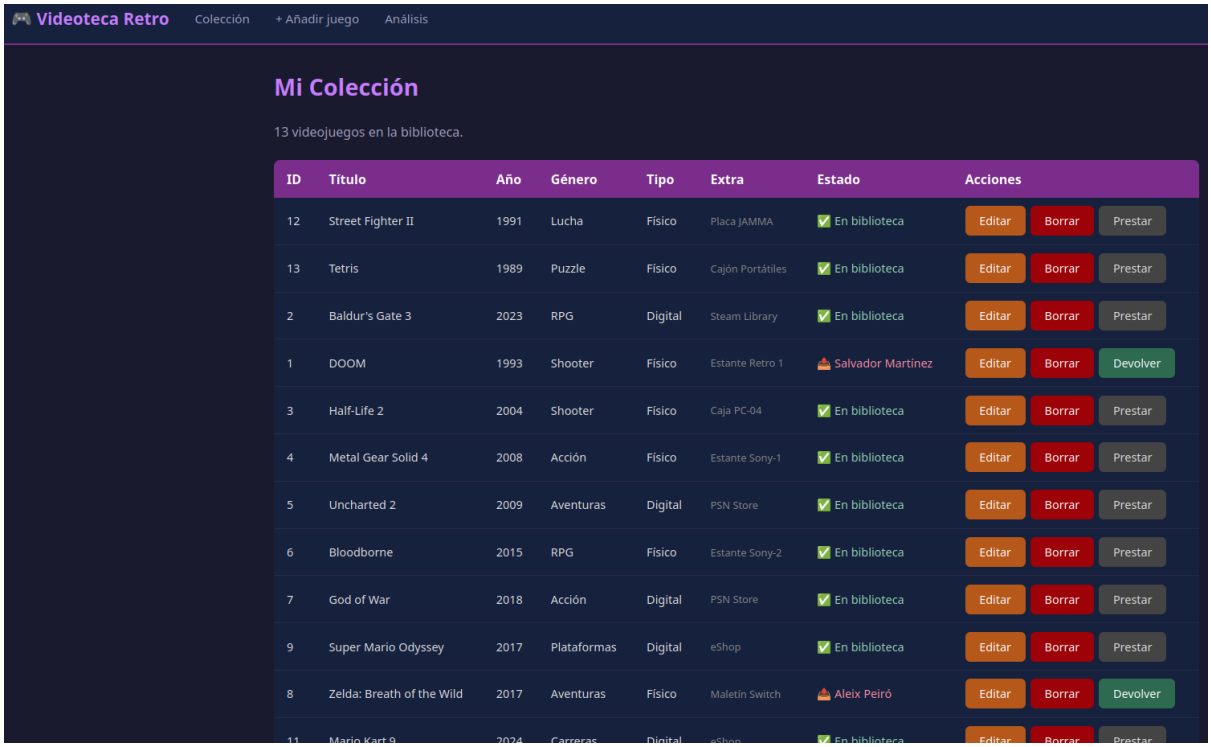
Las plantillas siguen el principio **DRY** (*Don't Repeat Yourself*) mediante el sistema de herencia de Django. La plantilla madre **`base.html`** define una única vez todos los elementos comunes: la barra de navegación, los estilos CSS, el bloque de mensajes flash y la estructura de página. El resto de las plantillas heredan de esta plantilla mediante **`{% extends 'catalogo/base.html' %}`** y solo definen su contenido específico dentro del bloque **`{% block contenido %}`**, sin duplicar ningún elemento estructural.

Los mensajes de éxito y error se gestionan mediante el framework de mensajes de Django (**`django.contrib.messages`**), que almacena notificaciones en la sesión del usuario y las consume automáticamente en la siguiente petición. Esto permite que tras una redirección POST/Redirect/GET el usuario vea el resultado de su acción ("Videojuego añadido correctamente") sin necesidad de pasar datos entre vistas.

Las cinco plantillas de contenido cubren todas las operaciones del sistema:

- **`coleccion.html:`** Vista principal con la tabla completa del inventario. Muestra el estado de préstamo de cada título con indicadores visuales diferenciados y expone los botones de acción (editar, borrar, prestar/devolver) en línea con cada registro.
- **`formulario_juego.html:`** Formulario reutilizable tanto para altas como para ediciones. Cuando se usa para editar, Django rellena automáticamente los campos con los valores actuales del registro, ofreciendo una experiencia de edición natural.
- **`confirmar_borrado.html:`** Página intermedia que muestra el nombre del juego a eliminar y requiere confirmación explícita antes de ejecutar el borrado.
- **`formulario_prestamo.html:`** Formulario minimalista para capturar el nombre del destinatario antes de registrar el préstamo.
- **`analisis.html:`** Muestra las tres gráficas EDA generadas por **Matplotlib**. Las imágenes se incrustan directamente en el HTML codificadas en base64, evitando la necesidad de escribir archivos en disco o gestionar rutas de archivos estáticos.

Captura de pantalla principal de la aplicación en el navegador web:



Capturas de pantallas de la aplicación web:

5. Capa de inteligencia de datos

Para aportar valor estratégico al coleccionista, la aplicación integra un módulo de análisis basado en el ecosistema de ciencia de datos de Python.

5.1. Limpieza y normalización de datos

El módulo **`AnalizadorColeccion`** aplica un pipeline de limpieza sobre el DataFrame antes de cualquier cálculo. Esta fase es necesaria porque los datos provienen de entrada manual del usuario, lo que introduce variabilidad inevitable: espacios extra al inicio o al final de un texto, valores nulos en registros incompletos o tipos de dato incorrectos tras la lectura desde SQLite.

El pipeline de limpieza ejecuta tres operaciones en orden:

```
df = df.dropna(subset=["plataforma", "anio", "genero"])  # Elimina filas incompletas
df["plataforma"] = df["plataforma"].str.strip()          # Normaliza espacios en texto
df["genero"]     = df["genero"].str.strip()
df["anio"]       = df["anio"].astype(int)                # Garantiza tipo entero
```

La primera operación elimina filas con valores nulos en las columnas críticas para el análisis, evitando que registros incompletos distorsionen las estadísticas. La segunda normaliza los campos de texto quitando espacios innecesarios, lo que previene que **`"NES"`** y **`"NES "`** se traten como plataformas distintas y fragmenten artificialmente los grupos. La tercera fuerza la conversión del año a entero, ya que SQLite puede devolver este campo como **`float`** en determinadas circunstancias tras operaciones de lectura con Pandas.

5.2. Visualización y hallazgos

La librería **Matplotlib** genera tres gráficos en una única figura con disposición horizontal (**`1 × 3`**), cada uno orientado a responder una pregunta distinta sobre la colección:

- **Distribución por plataforma** (gráfico de barras): Identifica qué sistemas concentran más títulos, revelando las preferencias históricas del coleccionista y posibles desequilibrios en la colección.
- **Evolución temporal acumulada** (gráfico de línea): Muestra el crecimiento de la colección a lo largo de los años, permitiendo identificar períodos de adquisición intensa o estancamiento.
- **Correlación género/año** (diagrama de dispersión): Permite observar tendencias históricas en los géneros predominantes por década, como el auge de los juegos de plataformas en los años 80 o los RPG en los 90.

En la interfaz web, las gráficas se generan en memoria mediante un buffer **`BytesIO`** y se codifican en base64, incrustándolas directamente en el HTML sin escribir ningún archivo en disco. Esto simplifica el despliegue y evita problemas de gestión de archivos temporales.

6. Calidad y verificación

La robustez de la aplicación es una prioridad técnica. El código sigue estrictamente el estándar PEP8 y utiliza docstrings detallados en todas las clases y métodos.

6.1. Batería de pruebas unitarias

Se ha implementado una suite de pruebas con **Pytest** organizada en cuatro ficheros, cada uno con una responsabilidad diferenciada:

- **`test_models.py`:** Verifica la lógica del modelo de dominio: que **`JuegoFisico`** y **`JuegoDigital`** se instancian correctamente, que el polimorfismo de **`mostrar_detalle()`** funciona, y que la validación de año futuro lanza **`ValueError`**.
- **`test_database.py`:** Prueba las operaciones CRUD sobre una base de datos de test temporal que se crea y destruye en cada ejecución mediante fixtures de Pytest, garantizando aislamiento total entre tests.
- **`test_analysis.py`:** Comprueba que el DataFrame cargado desde la BD no está vacío y que contiene las columnas esperadas con los tipos correctos.
- **`test_robustez.py`:** Verifica que el sistema se comporta correctamente ante entradas incorrectas: años futuros, tipos de datos erróneos y valores límite.

Resultado tras las correcciones aplicadas al código de los tests:

9/9 tests superados (100%).

6.2. Gestión de validación en la interfaz de consola

La clase **`InterfazConsola`** implementa una capa de validación propia mediante métodos privados reutilizables, de forma que ninguna entrada del usuario llega a la base de datos sin haber sido verificada previamente. Este enfoque centraliza la lógica de validación en un único lugar, evitando duplicarla en cada opción del menú.

Los tres métodos privados que componen esta capa son:

- **`_pedir_texto(campo)`**: Solicita un campo de texto en un bucle **`while True`** que solo termina cuando el usuario introduce un valor no vacío. Rechaza cadenas formadas únicamente por espacios mediante **`.strip()`**, impidiendo que se almacenen registros con campos en blanco que pasarían la validación superficial pero serían inútiles como datos.
- **`_pedir_anio()`**: Valida el año de lanzamiento en tres niveles: primero comprueba que la entrada sea un número entero mediante **`.isdigit()`**, evitando el **`ValueError`** que produciría un **`int(input(...))`** sin protección; después verifica que el año no sea anterior a 1970, descartando valores sin sentido en el contexto retro; finalmente comprueba que no supere el año actual, coherentemente con la misma restricción que aplica el modelo de dominio en **`_validar_anio()`**. Si alguna comprobación falla, informa al usuario del motivo concreto y vuelve a solicitar el dato.
- **`pedir_id(accion)`**: Valida que el identificador introducido sea un entero positivo mediante **`.isdigit()`** antes de convertirlo, eliminando el riesgo de que un texto o un cero provoquen un error en la consulta SQL posterior.

Esta arquitectura de validación en la capa de interfaz es complementaria, no sustitutiva, de la validación en el modelo. Ambas capas actúan de forma independiente: la interfaz filtra entradas malformadas antes de que lleguen al modelo, y el modelo aplica sus propias reglas de negocio independientemente de por dónde llegue el dato.

7. Conclusiones y líneas de mejora

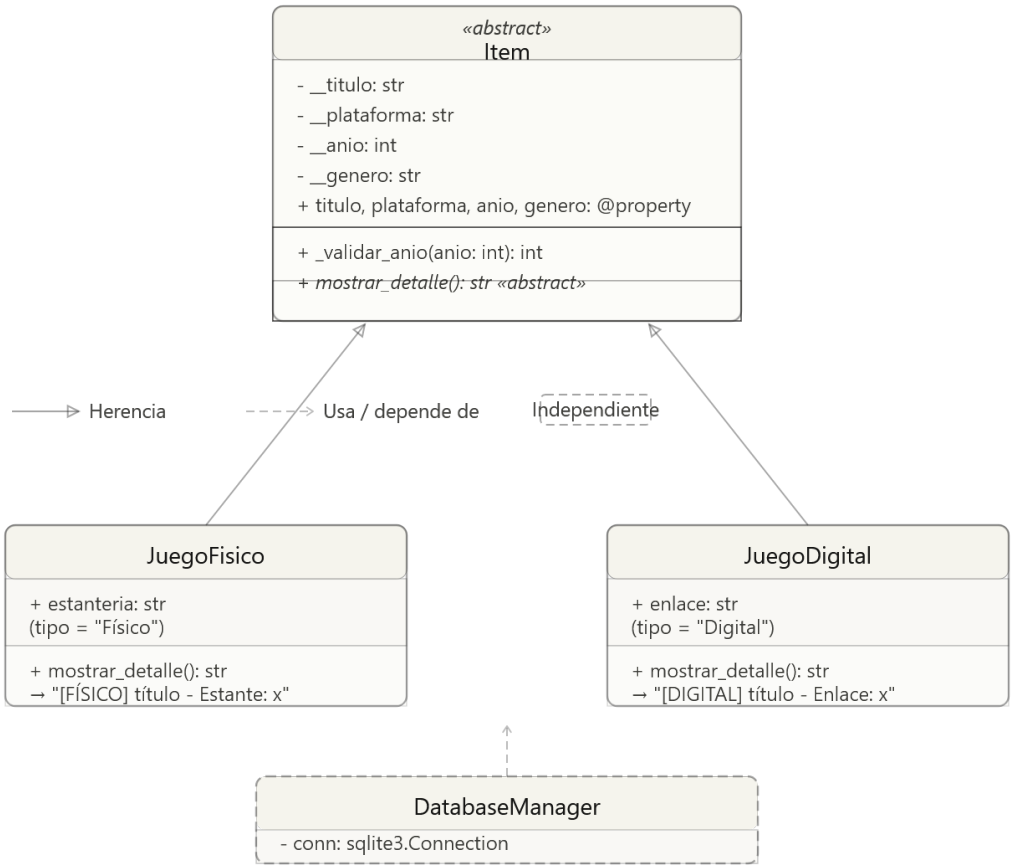
El proyecto culmina con un sistema funcional que cubre la totalidad de los requisitos establecidos en el enunciado: gestión CRUD completa, persistencia relacional con SQLite, interfaz web con Django, análisis exploratorio de datos y una suite de pruebas con cobertura del 100%. La arquitectura en capas adoptada (modelos, base de datos, vistas, plantillas y análisis) garantiza que cada módulo pueda evolucionar de forma independiente sin afectar al resto del sistema.

Como líneas de mejora para una versión de producción se proponen las siguientes:

- **Autenticación de usuarios:** Integrar Django Auth para proteger el acceso a la aplicación mediante login, permitiendo en el futuro gestionar múltiples colecciones de distintos usuarios de forma aislada.
- **Sincronización con APIs externas:** Conectar con bases de datos de videojuegos como IGDB o MobyGames para autocompletar metadatos (portada, descripción, puntuación) al introducir un título, reduciendo el esfuerzo de catalogación manual.
- **Notificaciones de préstamos:** Implementar un sistema de alertas por correo electrónico cuando un préstamo supere un número determinado de días, usando el módulo `smtplib` de Python o Celery para tareas programadas.
- **Despliegue en producción:** Sustituir el servidor de desarrollo de Django por Gunicorn como servidor WSGI, con Nginx como proxy inverso, permitiendo exponer la aplicación en red de forma segura y eficiente.
- **Importación masiva desde CSV:** Añadir una vista de importación que permita cargar colecciones completas desde un fichero CSV, aplicando el pipeline de limpieza ya existente en `AnalizadorColeccion`.

Este proyecto demuestra que la combinación de una arquitectura POO sólida, persistencia relacional bien diseñada y un framework web maduro como Django produce aplicaciones mantenibles, seguras y extensibles desde su primera versión.

Diagrama UML de clases



- `Item` en la parte superior con el estereotipo «`abstract`», sus 4 atributos privados (doble guion), las `@property` y el método abstracto `mostrar_detalle()`
- `JuegoFisico` y `JuegoDigital` heredando de `Item` (flecha con triángulo hueco), cada una con su atributo propio y su implementación concreta
- `DatabaseManager` abajo con borde discontinuo, conectado por una flecha de dependencia punteada
- Leyenda en el centro explicando los dos tipos de relación

Diagrama ERD

- **Tabla `JUEGOS`** con sus 7 columnas, indicando cuáles son `PK`, `NOT NULL` y qué valores admite `tipo`
- **Tabla `PRESTAMOS`** con `juego_id` marcado como `PK, FK` a la vez (eso es lo que garantiza la restricción 1:1)
- **Relación `||--o|`** entre ambas: un juego puede tener cero o un préstamo activo, nunca más de uno (notación crow's foot)

Nota: Los diagramas UML de clases y el esquema ERD se adjuntan como documentos independientes en el dossier digital del proyecto.